

# XRComm FCS Wideband Receive Platform

## Application Development Guide

XRComm Inc.

June 2026

# **XRComm FCS Wideband Receive Platform**

## Application Development Guide

|                       |                                      |
|-----------------------|--------------------------------------|
| <b>Platform</b>       | XRComm FCS Wideband Receive Platform |
| <b>Driver Version</b> | v1.4.3                               |
| <b>Variant</b>        | Wideband                             |
| <b>Audience</b>       | Software Developers                  |
| <b>Classification</b> | Proprietary                          |

© 2026 XRComm Inc. – All Rights Reserved.

## Contents

|          |                                                           |          |
|----------|-----------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                       | <b>3</b> |
| <b>2</b> | <b>Compiling and Running the Hello World Examples</b>     | <b>3</b> |
| 2.1      | Compile the Examples . . . . .                            | 3        |
| 2.2      | Run the IQ Example . . . . .                              | 3        |
| 2.3      | Run the Full-Packet Example . . . . .                     | 3        |
| 2.4      | Verify the Application Is Connected . . . . .             | 4        |
| <b>3</b> | <b>Application API Reference</b>                          | <b>5</b> |
| 3.1      | Quick API Use Case: Consume Wideband IQ Samples . . . . . | 5        |
| 3.2      | Lifecycle and Connection Functions . . . . .              | 5        |
| 3.2.1    | xrcomm_ip_client_init . . . . .                           | 5        |
| 3.2.2    | xrcomm_ip_client_init_port . . . . .                      | 5        |
| 3.2.3    | xrcomm_ip_client_wait_ready . . . . .                     | 6        |
| 3.2.4    | xrcomm_ip_client_running . . . . .                        | 6        |
| 3.2.5    | xrcomm_ip_client_shutdown . . . . .                       | 6        |
| 3.3      | Data Consumer Functions . . . . .                         | 6        |
| 3.3.1    | xrcomm_ip_client_poll . . . . .                           | 6        |
| 3.3.2    | xrcomm_ip_client_consume . . . . .                        | 6        |
| 3.4      | Diagnostics and Telemetry . . . . .                       | 6        |
| 3.4.1    | xrcomm_ip_client_get_dropped . . . . .                    | 6        |
| 3.4.2    | xrcomm_ip_client_report_spectrum . . . . .                | 6        |

## 1 Introduction

This document provides detailed guidelines for developers writing, compiling, and running custom applications that interface with the XRComm FCS Wideband Platform.

The platform provides a header-only, zero-copy C API (`xrcomm_ip_client.h`) for POSIX applications that consume I/Q sample batches, and supports DPDK secondary process attachment for full-packet processing applications. IQ applications do not link against DPDK or SPDK; those libraries are used by the platform services. Reference documentation is available from DPDK (<https://core.dpdk.org/doc/>) and SPDK (<https://spdk.io/doc/>).

## 2 Compiling and Running the Hello World Examples

The platform includes two example consumer applications in the `wideband/hello_world/` folder that demonstrate how to subscribe to and consume platform data streams.

| Example Binary                   | Delivery Mode                        | Enable With                 | Privileges        |
|----------------------------------|--------------------------------------|-----------------------------|-------------------|
| <code>hello_world</code>         | IQ samples (zero-copy shared memory) | <code>set-ip on</code>      | None              |
| <code>hello_world_fullpkt</code> | Full packets (DPDK secondary)        | <code>set-fullpkt on</code> | <code>sudo</code> |

### 2.1 Compile the Examples

From the `XRComm_FCS_platform` root directory:

```
cd wideband/hello_world
mkdir -p build && cd build
cmake ..
make
cd ../../
```

Compiled binaries are placed in `wideband/hello_world/bin/`.

### 2.2 Run the IQ Example

The IQ example connects to the Platform Driver via zero-copy shared memory. It prints a rising count of received batches and calculated power levels (dBFS).

#### Terminal 1 (FlexCompute Server):

```
./wideband/hello_world/bin/hello_world
```

#### Terminal 2 (Client Computer or FlexCompute Server):

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-ip on
```

### 2.3 Run the Full-Packet Example

The full-packet example attaches to the Platform Driver's memory pool as a DPDK secondary process to consume raw Ethernet frames. It requires root privileges.

#### Terminal 1 (FlexCompute Server):

```
sudo ./wideband/hello_world/bin/hello_world_fullpkt
```

#### Terminal 2 (Client Computer or FlexCompute Server):

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-fullpkt on
```

## 2.4 Verify the Application Is Connected

Query the registry from the **Client Computer**:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 list-ips
```

```
# Output: slot=0 hello_world mode=IQ active
```

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 get-mode-stats
```

```
# Output: per-application received / processed / dropped counts
```

### 3 Application API Reference

Include `<xrcomm/xrcomm_ip_client.h>` in your C/C++ applications. The header uses standard POSIX shared memory (`shm_open`, `mmap`) and does not require linking against DPDK libraries.

#### 3.1 Quick API Use Case: Consume Wideband IQ Samples

The usual customer application registers one IQ consumer, waits for the shared-memory ring, polls available batches, processes the interleaved int16 IQ samples, and releases each batch:

```
#include <stdio.h>
#include <xrcomm/xrcomm_ip_client.h>

int main(void) {
    xrcomm_ip_client_t client;
    if (xrcomm_ip_client_init(&client, "wideband_iq_app") != 0) {
        return 1;
    }
    if (xrcomm_ip_client_wait_ready(&client, 5000) != 0) {
        xrcomm_ip_client_shutdown(&client);
        return 1;
    }

    while (xrcomm_ip_client_running(&client)) {
        const xrcomm_iq_slot_t *slot = xrcomm_ip_client_poll(&client);
        if (!slot) {
            continue;
        }

        printf("samples=%u dropped=%lu\n",
            slot->valid_sample_count,
            xrcomm_ip_client_get_dropped(&client));
        xrcomm_ip_client_report_spectrum(&client, 1, 0.0f);
        xrcomm_ip_client_consume(&client);
    }

    xrcomm_ip_client_shutdown(&client);
    return 0;
}
```

Enable IQ delivery from the gRPC client with `set-ip` on before expecting batches.

#### 3.2 Lifecycle and Connection Functions

##### 3.2.1 `xrcomm_ip_client_init`

```
int xrcomm_ip_client_init(xrcomm_ip_client_t *client, const char *name);
```

Registers the application with the platform under the specified name and maps the primary control region. Returns 0 on success.

##### 3.2.2 `xrcomm_ip_client_init_port`

```
int xrcomm_ip_client_init_port(xrcomm_ip_client_t *client, const char *name,
    uint32_t hsp_port, uint32_t channel);
```

Registers the application and requests a specific NIC port (0 or 1) and channel (0..3, or 255 for all channels combined).

### 3.2.3 `xrcomm_ip_client_wait_ready`

```
int xrcomm_ip_client_wait_ready(xrcomm_ip_client_t *client, int timeout_ms);
```

Blocks the caller until the Platform Driver has created and opened the shared memory ring buffer for this application.

### 3.2.4 `xrcomm_ip_client_running`

```
bool xrcomm_ip_client_running(xrcomm_ip_client_t *client);
```

Returns `true` as long as the platform's main receive loop is active. Use this function as the loop guard for sample ingestion.

### 3.2.5 `xrcomm_ip_client_shutdown`

```
void xrcomm_ip_client_shutdown(xrcomm_ip_client_t *client);
```

Deregisters the application from the platform and unmaps/unlinks all shared memory regions.

## 3.3 Data Consumer Functions

### 3.3.1 `xrcomm_ip_client_poll`

```
const xrcomm_iq_slot_t* xrcomm_ip_client_poll(xrcomm_ip_client_t *client);
```

Polls the application ring buffer for a new batch of IQ samples. Returns a read-only pointer to the sample slot if data is available, or `NULL` if the ring is empty.

### 3.3.2 `xrcomm_ip_client_consume`

```
void xrcomm_ip_client_consume(xrcomm_ip_client_t *client);
```

Releases the last slot returned by `poll()`, advancing the read pointer and returning the slot to the Platform Driver's write pool. Must be called exactly once after a successful poll.

## 3.4 Diagnostics and Telemetry

### 3.4.1 `xrcomm_ip_client_get_dropped`

```
uint64_t xrcomm_ip_client_get_dropped(xrcomm_ip_client_t *client);
```

Returns the number of sample batches dropped by the Platform Driver since connection start due to the application's ring buffer being full.

### 3.4.2 `xrcomm_ip_client_report_spectrum`

```
void xrcomm_ip_client_report_spectrum(xrcomm_ip_client_t *client,  
                                     uint64_t processed, float power_dbfs);
```

Reports the application's processing metrics (total batches consumed and current signal power) back to the Platform Driver. These values are displayed in the gRPC telemetry and statistics.